Validation

Season: 6 · FP Foundations · Category: Functional · Level: Intermediate

Intent

- Accumulate all errors, not just the first.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Form validation highlighting every bad field.

When to Use

- Pure functional style in Java 25 + Zazr.
- The design force named in Intent shows up more than once.

When Not to Use

- Throwing exceptions everywhere is 'fine' for a script.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.functional.validation;

import dev.zazr.control.Validation;

record Signup(String email, int age) {}

public final class SignupValidation {
    private SignupValidation() {}

    public static Validation<String, String> email(String raw) {
        return Validation.fromPredicate(raw, e -> e != null && e.contains("@"), e -> "email has no @");
    }

    public static Validation<String, Integer> age(int raw) {
        return Validation.fromPredicate(raw, a -> a >= 18, a -> "age under 18");
    }

    public static Validation<String, Signup> signup(String email, int age) {
        return Validation.zipWith(email(email), age(age), Signup::new);
    }

    public static String message(Validation<String, Signup> v) {
        return switch (v) {
            case Validation.Valid(var s) -> "welcome " + s.email();
            case Validation.Invalid(var errors) -> errors.mkString(", ");
        };
    }
}
```

Modern Java 25 / Functional

- Primary examples use Zazr; VAVR shown in comparison episode.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: functional/validation/
```

Pitfalls

- Zazr rejects null inside Option/collections.
- Keep interpreters at the edge.

Interview Q&A

- Q: Interview: what problem does Validation solve? A: Accumulate all errors, not just the first.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Accumulate all errors, not just the first.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Form validation highlighting every bad field. [0:40] Intent — State the design force in one sentence...